

Database Management System

CASE STUDY 44

Hostel Room Allotment and Mess Billing

Hostel Room Allotment and Mess Billing Database Using SQL

Prepared by

Student Name: Lokendra Joshi

Roll No.: 150096725062

Cohort: Larry Page

Academic Year: 2025-2029

ITM Skills University

Index

No.	Section	Page
1	Title	3
2	Problem Statement	3
3	Objectives	4
4	Sample Data Used	4
5	Tools and Environment	5
6	Database Design	5
7	SQL Solution with Screenshots	7
8	Results	15
9	Conclusion	16
10	Declaration	16

The page numbers above follow the report structure and may shift slightly when the document is opened or printed with a different Word/PDF rendering engine.

1. Title

Hostel Room Allotment and Mess Billing Database Using SQL

Case Study 44 — a PostgreSQL-based relational database for managing hostel blocks, rooms, student allocations, mess attendance and monthly mess billing. The case study focuses on practical SQL, relational design, constraints, indexing and execution-plan analysis.

2. Problem Statement

A college hostel allocates rooms to students, records mess attendance, and generates mess bills based on actual meal consumption. The database must support day-to-day hostel operations while preserving allocation history and making vacated rooms immediately available for re-allotment.

The system must answer operational questions such as: (1) How many rooms are vacant in each block? (2) What is each student's mess bill this month? (3) Which students skipped more than ten meals this month? (4) What is the occupancy rate of each block? (5) Which rooms were vacated but have not been re-allotted?

The design also requires realistic data volume and an indexing experiment using EXPLAIN ANALYZE. The project therefore evaluates sequential scans, index scans, composite-index column order, and the situation in which adding an index can make a query slower.

3. Objectives

The objectives of the case study are:

- Model blocks, rooms, room allocations, students, mess attendance and bills so that allocation history is preserved and vacated rooms can be re-allotted.
- Use primary keys, foreign keys, UNIQUE constraints, CHECK constraints and a partial unique index to enforce important business rules.
- Apply filtering and sorting, aggregate functions with GROUP BY and HAVING, joins and subqueries.
- Seed the database with realistic data volume rather than a few demonstration rows.
- Create and evaluate single-column and composite indexes using EXPLAIN ANALYZE.

- Demonstrate at least one query where adding an index makes execution slower and explain the reason.
- Study how composite-index column order affects index usability.
- Record design decisions and explain reasonable alternatives that were rejected.

4. Sample Data Used

The database was seeded with a realistic test dataset. The final clean rebuild produced the following record counts:

Table	Records	Purpose
blocks	8	Hostel blocks A to H
rooms	240	30 rooms per block
students	400	Student master records
room_allocations	400	100 historical + 300 current allocations
meal_types	3	Breakfast, Lunch and Dinner
mess_attendance	36,000	400 students × 30 days × 3 meals
mess_bills	400	One September 2026 bill per student
mess_bill_items	1,200	Three meal-level items per bill

Meal rates used in the seed data were Breakfast ₹30, Lunch ₹50 and Dinner ₹60. Attendance covers September 1–30, 2026. Students 1–20 were intentionally given more than ten skipped meals so that the HAVING query has a meaningful result.

Room capacities are four beds by default, with every tenth room having capacity six. The allocation data contains both historical records with a vacated_on date and current records where vacated_on is NULL.

5. Tools and Environment

Item	Details
Database Management System	PostgreSQL 18.4
Client	psql command-line terminal on macOS
Database Name	hostel_db
Language	SQL (DDL, DML and analytical queries)
Analysis Tool	EXPLAIN (ANALYZE, BUFFERS)
Project Files	schema.sql, seed.sql, indexes.sql, queries.sql, explain_analyze.sql, README.md

The database scripts were also executed against a clean test database named `hostel_db_final_test` using `ON_ERROR_STOP=1`. This clean rebuild verified that the schema, seed data, final indexes and required queries execute successfully from scratch.

6. Database Design

The database contains eight related tables. The design separates master data, allocation history, attendance records and billing details. Foreign keys maintain relationships between entities, while constraints prevent invalid states.

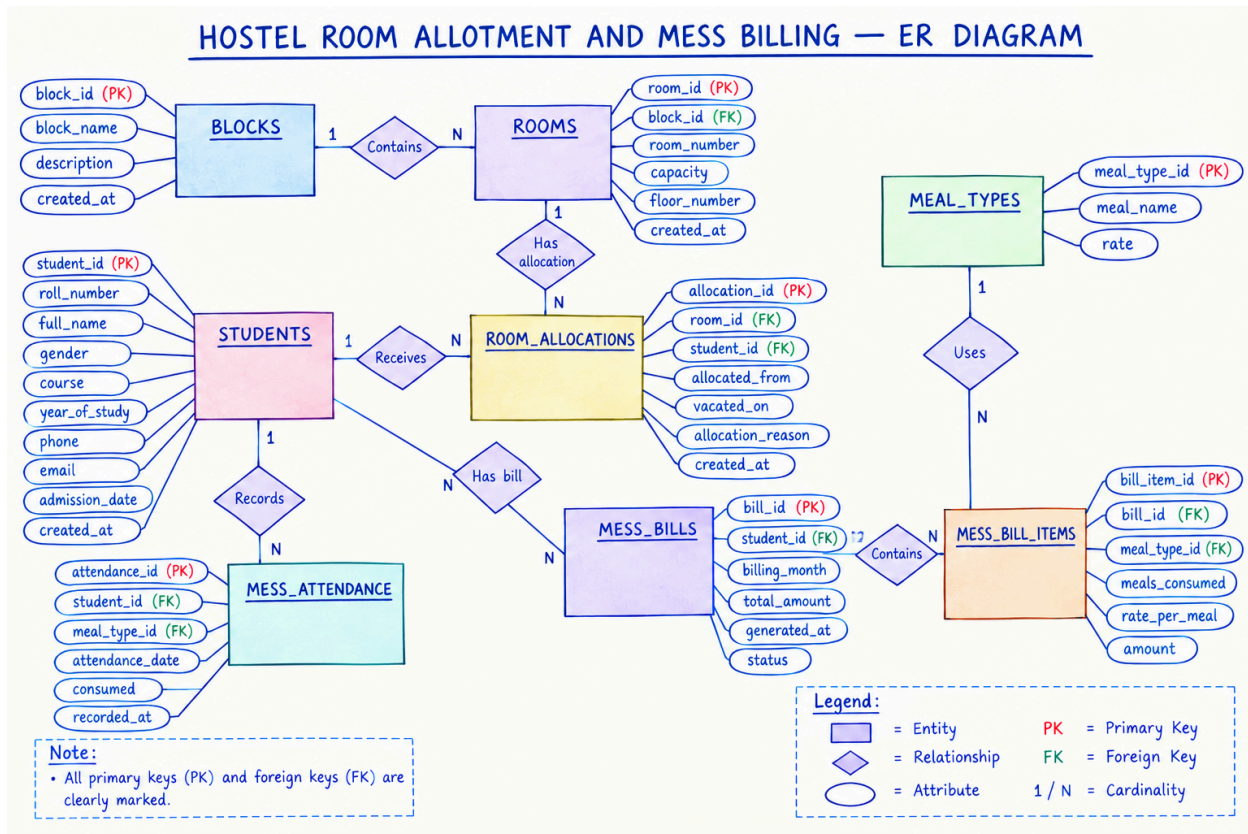


Figure 1: Entity-Relationship diagram for the hostel room allotment and mess billing database.

6.1 Table: blocks

Column	Data Type	Constraint
block id	BIGSERIAL	PRIMARY KEY
block name	VARCHAR(50)	NOT NULL, UNIQUE
description	TEXT	Optional
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT TIMESTAMP

6.2 Table: rooms

Column	Data Type	Constraint
room id	BIGSERIAL	PRIMARY KEY
block id	BIGINT	NOT NULL, FK → blocks
room number	VARCHAR(20)	NOT NULL
capacity	INTEGER	NOT NULL, CHECK > 0
floor number	INTEGER	NOT NULL, CHECK > 0
created_at	TIMESTAMP	DEFAULT CURRENT TIMESTAMP

6.3 Table: students

Column	Data Type	Constraint
student_id	BIGSERIAL	PRIMARY KEY
roll_number	VARCHAR(30)	NOT NULL, UNIQUE
full_name	VARCHAR(100)	NOT NULL
gender	VARCHAR(20)	CHECK: Male/Female/Other
course	VARCHAR(100)	NOT NULL
year_of_study	INTEGER	CHECK: 1–6
phone / email	VARCHAR	Optional; email UNIQUE
admission_date	DATE	NOT NULL

6.4 Table: room_allocations

Column	Data Type	Constraint
allocation_id	BIGSERIAL	PRIMARY KEY
room_id	BIGINT	NOT NULL, FK → rooms
student_id	BIGINT	NOT NULL, FK → students
allocated_from	DATE	NOT NULL
vacated_on	DATE	NULL for current allocation
allocation_reason	VARCHAR(100)	Optional
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

A partial unique index on room_allocations(student_id) WHERE vacated_on IS NULL ensures that a student cannot have more than one active room allocation. Historical allocations remain available for reporting.

6.5 Table: meal_types

Column	Data Type	Constraint
meal_type_id	SMALLSERIAL	PRIMARY KEY
meal_name	VARCHAR(30)	NOT NULL, UNIQUE
rate	NUMERIC(10,2)	NOT NULL, CHECK ≥ 0

6.6 Table: mess_attendance

Column	Data Type	Constraint
attendance_id	BIGSERIAL	PRIMARY KEY
student_id	BIGINT	NOT NULL, FK → students
meal_type_id	SMALLINT	NOT NULL, FK → meal types
attendance_date	DATE	NOT NULL

consumed	BOOLEAN	NOT NULL, DEFAULT TRUE
recorded_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

6.7 Table: mess_bills

Column	Data Type	Constraint
bill_id	BIGSERIAL	PRIMARY KEY
student_id	BIGINT	NOT NULL, FK → students
billing_month	DATE	NOT NULL
total_amount	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0
generated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
status	VARCHAR(20)	Generated / Paid / Pending

6.8 Table: mess_bill_items

Column	Data Type	Constraint
bill_item_id	BIGSERIAL	PRIMARY KEY
bill_id	BIGINT	NOT NULL, FK → mess_bills
meal_type_id	SMALLINT	NOT NULL, FK → meal_types
meals_consumed	INTEGER	CHECK ≥ 0
rate_per_meal	NUMERIC(10,2)	CHECK ≥ 0
amount	NUMERIC(12,2)	CHECK ≥ 0

The main relationships are one-to-many: one block contains many rooms; one room can have many historical allocations; one student can have many historical allocations, attendance records and bills; one bill contains multiple bill items; and one meal type can be referenced by many attendance and bill-item records.

The design intentionally stores room allocation history rather than overwriting a student's previous allocation. This allows a room to be identified as currently occupied when vacated_on IS NULL, while preserving previous occupancy records.

7. SQL Solution with Screenshots

7.1 Create and connect to the database

A dedicated PostgreSQL database was created and the session was switched to it before the project schema was loaded.

```
CREATE DATABASE hostel_db;
\c hostel_db
```

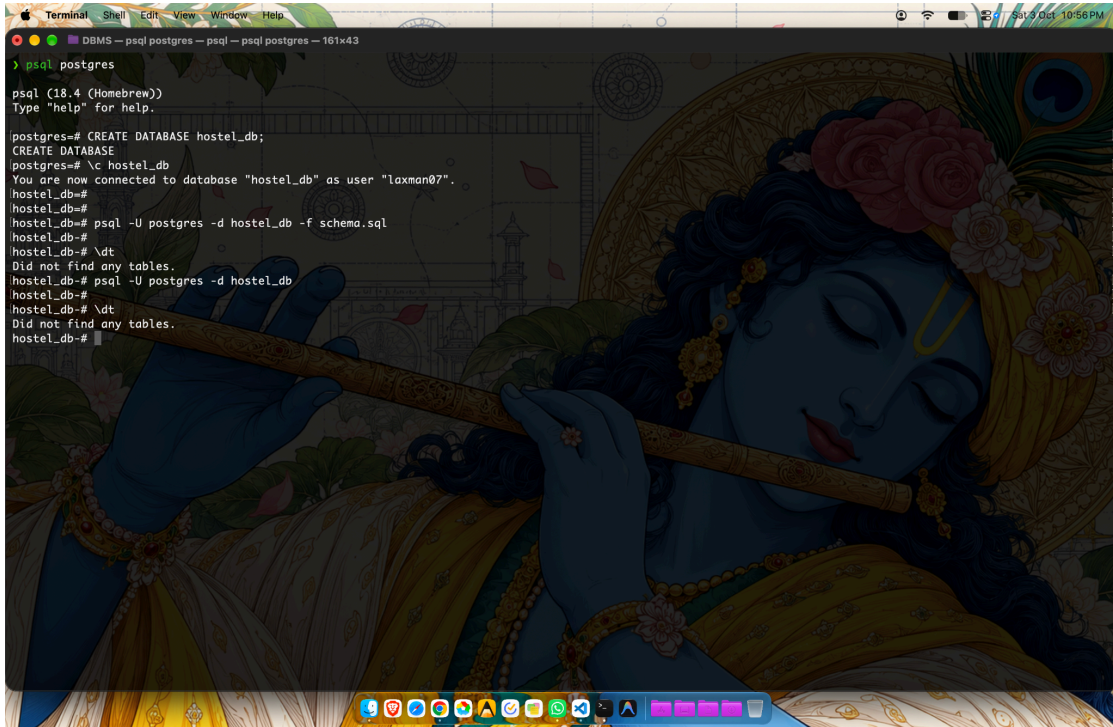


Figure 2: Creating and connecting to the hostel database.

7.2 Create the tables and constraints

The schema script creates the eight tables, foreign-key relationships, uniqueness rules and CHECK constraints. It also creates the partial unique index that prevents multiple active allocations for the same student.

```
CREATE TABLE blocks (
    block_id BIGSERIAL PRIMARY KEY,
    block_name VARCHAR(50) NOT NULL UNIQUE,
    description TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE rooms (
    room_id BIGSERIAL PRIMARY KEY,
    block_id BIGINT NOT NULL REFERENCES blocks(block_id)
        ON DELETE RESTRICT,
    room_number VARCHAR(20) NOT NULL,
    capacity INTEGER NOT NULL DEFAULT 4,
    floor_number INTEGER NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_room_block_number UNIQUE (block_id, room_number)
```

```
);
```

```
CREATE UNIQUE INDEX uq_active_student_allocation
ON room_allocations(student_id)
WHERE vacated_on IS NULL;
```

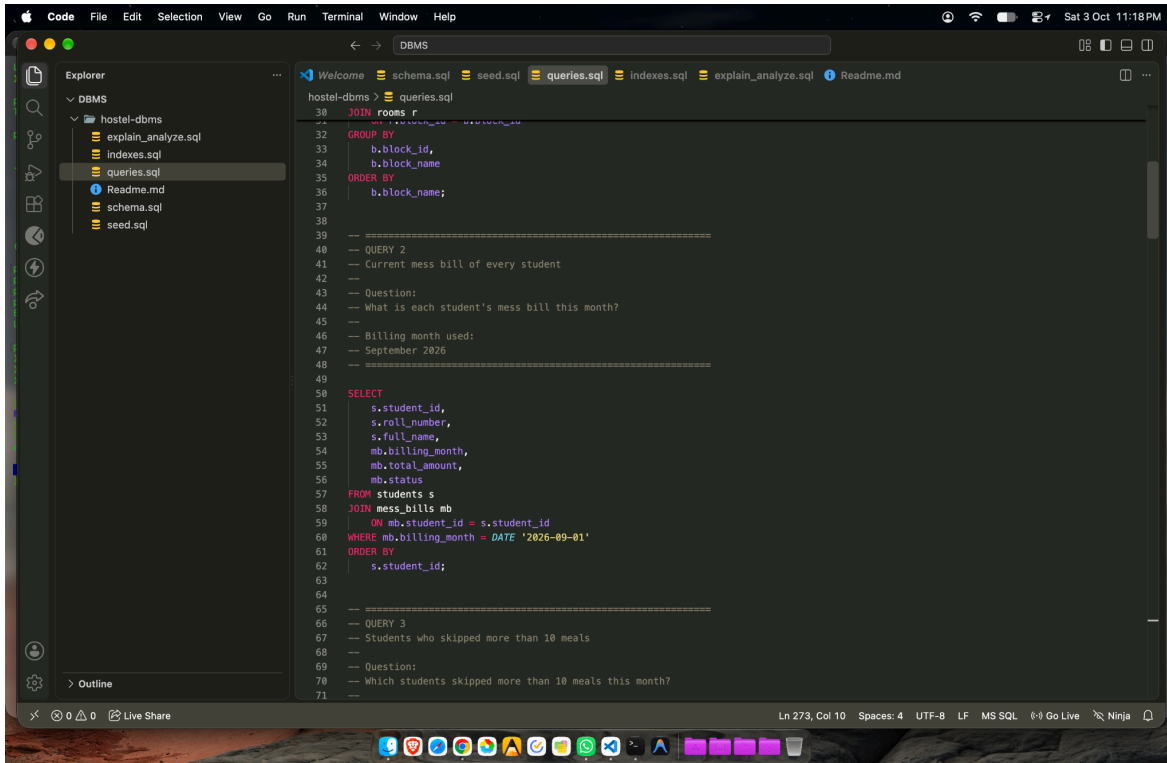


Figure 3: Project schema and SQL definitions in the development environment.

7.3 Insert the sample data

The seed script inserts the eight blocks, 240 rooms, 400 students, three meal types, 400 room allocations, 36,000 attendance records, 400 bills and 1,200 bill items. The final clean rebuild reported successful INSERT operations for all datasets.

```
INSERT 0 8
INSERT 0 240
INSERT 0 400
INSERT 0 3
INSERT 0 100
INSERT 0 300
INSERT 0 36000
INSERT 0 400
INSERT 0 1200
ANALYZE
```

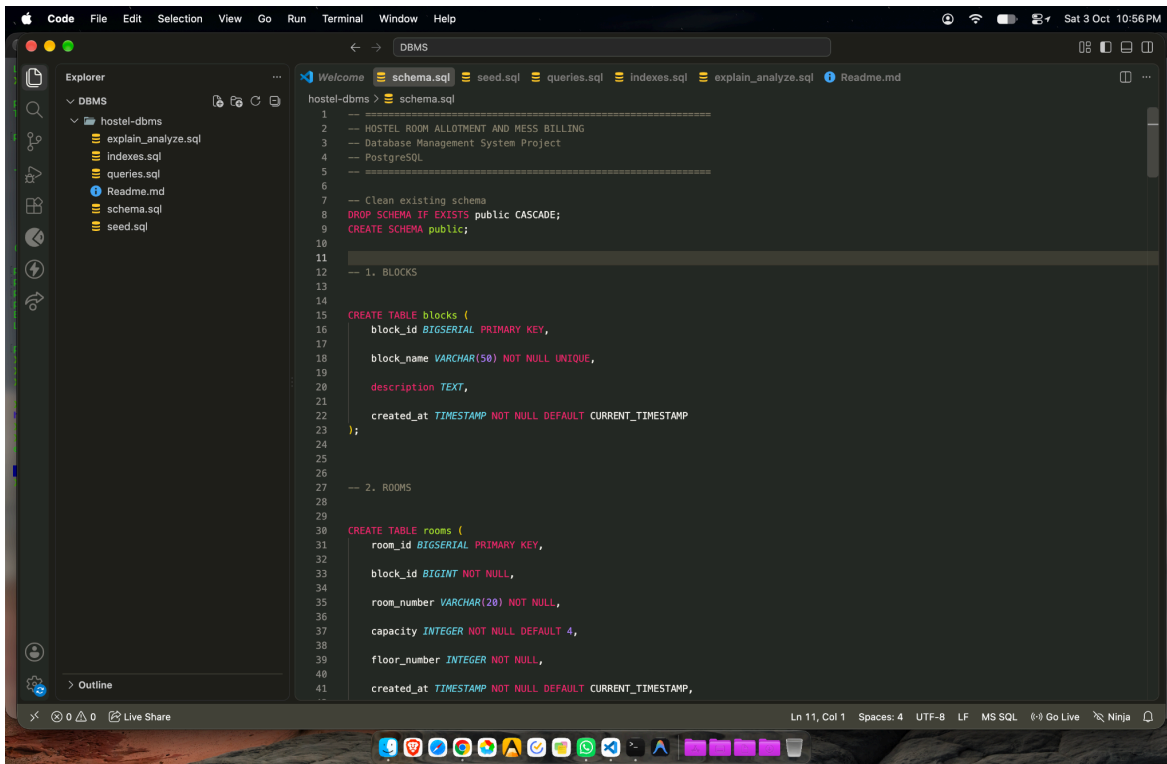


Figure 4: Seed-data generation and SQL script execution.

7.4 Find vacant rooms in each block

A room is counted as vacant when there is no active allocation for that room. The NOT EXISTS subquery checks for an allocation whose vacated_on is NULL.

```

SELECT
    b.block_name,
    COUNT(r.room_id) AS total_rooms,
    COUNT(r.room_id) FILTER (
        WHERE NOT EXISTS (
            SELECT 1
            FROM room_allocations ra
            WHERE ra.room_id = r.room_id
              AND ra.vacated_on IS NULL
        )
    ) AS vacant_rooms
FROM blocks b
JOIN rooms r ON r.block_id = b.block_id
GROUP BY b.block_id, b.block_name
ORDER BY b.block_name;
  
```

Final result: A, B and C Blocks each have 20 vacant rooms. D, E, F, G and H Blocks each have 21 vacant rooms.

7.5 Display each student's September 2026 mess bill

```
SELECT
    s.student_id,
    s.roll_number,
    s.full_name,
    mb.billing_month,
    mb.total_amount,
    mb.status
FROM students s
JOIN mess_bills mb ON mb.student_id = s.student_id
WHERE mb.billing_month = DATE '2026-09-01'
ORDER BY s.student_id;
```

The query joins students with mess_bills and returns one September 2026 bill for each of the 400 students. Bill status is retained as Generated, Paid or Pending.

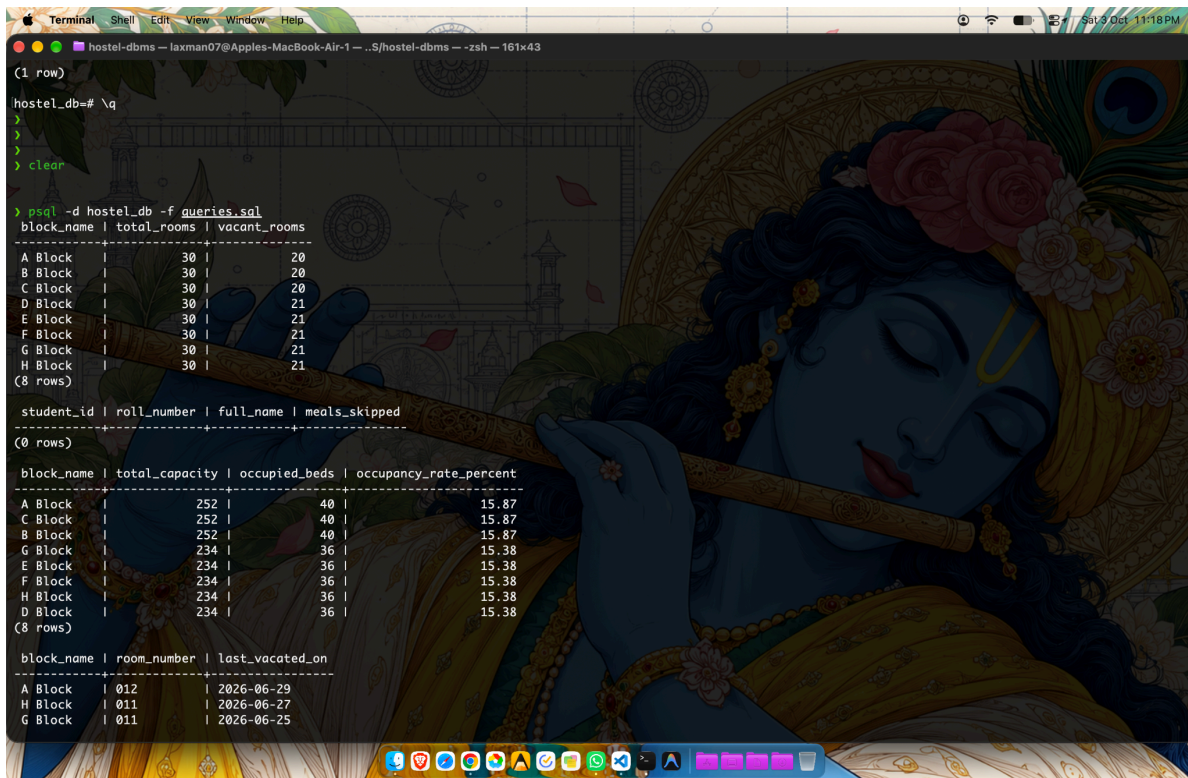


Figure 5: SQL execution and result output for the hostel project queries.

7.6 Find students who skipped more than ten meals

```
SELECT
    s.student_id,
    s.roll_number,
    s.full_name,
    COUNT(*) AS meals_skipped
```

```

FROM students s
JOIN mess_attendance ma ON ma.student_id = s.student_id
WHERE ma.attendance_date >= DATE '2026-09-01'
      AND ma.attendance_date < DATE '2026-10-01'
      AND ma.consumed = FALSE
GROUP BY s.student_id, s.roll_number, s.full_name
HAVING COUNT(*) > 10
ORDER BY meals_skipped DESC, s.student_id;

```

The GROUP BY and HAVING clauses identify students whose number of skipped meals exceeds ten. The seed data intentionally gives students 1–20 more than ten skipped meals, providing a controlled test case for the query.

7.7 Calculate occupancy rate of each block

```

SELECT
    b.block_name,
    SUM(r.capacity) AS total_capacity,
    COUNT(ra.student_id) AS occupied_beds,
    ROUND(
        COUNT(ra.student_id) * 100.0
        / NULLIF(SUM(r.capacity), 0),
        2
    ) AS occupancy_rate_percent
FROM blocks b
JOIN rooms r ON r.block_id = b.block_id
LEFT JOIN room_allocations ra
    ON ra.room_id = r.room_id
    AND ra.vacated_on IS NULL
GROUP BY b.block_id, b.block_name
ORDER BY occupancy_rate_percent DESC;

```

The occupancy calculation uses active room allocations only. Total capacity is the sum of room capacities in the block, while occupied beds are counted from current allocations.

7.8 Find rooms that were vacated but not re-allotted

```

SELECT
    b.block_name,
    r.room_number,
    MAX(ra.vacated_on) AS last_vacated_on
FROM blocks b
JOIN rooms r ON r.block_id = b.block_id
JOIN room_allocations ra ON ra.room_id = r.room_id

```

```

WHERE ra.vacated_on IS NOT NULL
AND NOT EXISTS (
    SELECT 1
    FROM room_allocations current_ra
    WHERE current_ra.room_id = r.room_id
    AND current_ra.vacated_on IS NULL
)
GROUP BY b.block_id, b.block_name, r.room_id, r.room_number
ORDER BY last_vacated_on DESC;

```

The final clean test returned 25 historical rooms that had been vacated and did not currently have an active allocation.

7.9 Additional reporting queries

The project also includes supporting queries for current student-room details, bill status, total September revenue, meal consumption and the ten highest mess bills. These demonstrate joins, filtering, aggregation, ordering and reporting use cases.

```

-- Total September revenue
SELECT
    billing_month,
    COUNT(*) AS total_bills,
    SUM(total_amount) AS total_mess_revenue,
    ROUND(AVG(total_amount), 2) AS average_bill
FROM mess_bills
WHERE billing_month = DATE '2026-09-01'
GROUP BY billing_month;

-- Meal consumption summary
SELECT
    mt.meal_name,
    COUNT(*) FILTER (WHERE ma.consumed = TRUE) AS meals_consumed,
    COUNT(*) FILTER (WHERE ma.consumed = FALSE) AS meals_skipped
FROM meal_types mt
JOIN mess_attendance ma
    ON ma.meal_type_id = mt.meal_type_id
WHERE ma.attendance_date >= DATE '2026-09-01'
    AND ma.attendance_date < DATE '2026-10-01'
GROUP BY mt.meal_type_id, mt.meal_name
ORDER BY mt.meal_type_id;

```

The verified September 2026 revenue was ₹15,35,360.00 and the average bill was ₹3,838.40.

7.10 Indexing and EXPLAIN ANALYZE

Indexes were tested experimentally rather than assuming that every index would improve performance. The final retained date index supports attendance queries filtered by `attendance_date`.

```
CREATE INDEX idx_attendance_date
ON mess_attendance (attendance_date);

ANALYZE mess_attendance;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    ma.attendance_date,
    COUNT(*) AS total_records,
    COUNT(*) FILTER (WHERE ma.consumed = TRUE) AS meals_consumed,
    COUNT(*) FILTER (WHERE ma.consumed = FALSE) AS meals_skipped
FROM mess_attendance ma
WHERE ma.attendance_date = DATE '2026-09-15'
GROUP BY ma.attendance_date;
```

```

-- 7. MESS ATTENDANCE
--
-- 400 students
-- x 30 days
-- x 3 meals
-- = 36,000 attendance records
--
-- Approximately 8% of meals are skipped.

INSERT INTO mess_attendance (
    student_id,
    meal_type_id,
    attendance_date,
    consumed
)
SELECT
    s.student_id,
    m.meal_type_id,
    d.attendance_date,

    CASE
        WHEN (
            (s.student_id * 31)
            + (EXTRACT(
                DAY FROM d.attendance_date
            )::INTEGER * 17)
            + (m.meal_type_id * 13)
        ) % 100 < 8
        THEN FALSE
        ELSE TRUE
    END

FROM generate_series(
    DATE '2026-09-01',
    DATE '2026-09-30',
    INTERVAL '1 day'
) AS d(attendance_date)

CROSS JOIN (
    SELECT student_id
    FROM students
) AS s

CROSS JOIN meal_types AS m;

```

Figure 6: EXPLAIN ANALYZE/index experiment in the PostgreSQL environment.

7.11 Index experiment results

Experiment	Before	After	Interpretation
Attendance by date	Seq Scan; 3.937 ms	Index Scan; 0.372 ms	Successful index; approximately 10.6× faster in the documented run.
Skipped meals	Seq Scan; 4.463 ms	Bitmap scan; 5.695 ms	Index made this run slower because the table is modest and many rows matched.

Bills by month	Seq Scan; 1.571 ms	Seq Scan; 0.793 ms	Index not selected because the query matched the entire 400-row table.
Wrong composite order	Seq Scan; 4.757 ms	Seq Scan; 7.045 ms	Index not used because student_id was the leading column but was not filtered.
Room/vacated index	Seq Scan; 1.226 ms	Seq Scan; 0.635 ms	Index not selected; room_allocations contains only 400 rows.

The skipped-meals experiment is the required example where adding an index made the query worse. The reason is not that indexes are inherently bad; rather, the query returned a relatively large fraction of a 36,000-row table, so index traversal plus heap access was more expensive than scanning the table sequentially.

The composite index experiment also demonstrates column order. An index on (student_id, attendance_date, consumed) is naturally useful when student_id is constrained first. It was not useful for a query that filtered only attendance_date and consumed because the leading student_id column was unconstrained.

8. Results

Every major operation required by the case study was executed successfully in PostgreSQL. The final clean rebuild confirmed that all four main project scripts execute successfully from a fresh database.

No.	Operation	Verified Result
1	Database and schema creation	8 tables created successfully
2	Data seeding	8 blocks, 240 rooms, 400 students, 400 allocations, 36,000 attendance rows, 400 bills and 1,200 bill items
3	Vacant rooms	20 vacant rooms in A–C; 21 vacant rooms in D–H
4	September bills	400 student bills generated
5	Skipped meals	Query returns students with more than 10 skipped meals
6	Occupancy	Block-level occupied beds and occupancy percentages calculated
7	Vacated rooms	25 vacated rooms not currently re-allotted
8	September revenue	₹15,35,360.00

9	Average bill	₹3,838.40
10	Successful index test	Attendance-date query changed from Seq Scan to Index Scan

Meal-level verification showed 10,966 consumed and 1,034 skipped Breakfast records; 10,966 consumed and 1,034 skipped Lunch records; and 10,968 consumed and 1,032 skipped Dinner records.

The final database therefore demonstrates both functional correctness and the intended indexing concepts: an index can be highly beneficial for a selective date filter, while low-selectivity queries or small tables can continue to favor sequential scans.

9. Conclusion

This case study demonstrated how a hostel room-allotment and mess-billing system can be modeled using a relational PostgreSQL database. The eight-table design separates hostel structure, student information, allocation history, attendance and billing while maintaining relationships through foreign keys.

The room allocation history design is important because a vacated room must become available without deleting the previous allocation record. The partial unique index on active allocations also prevents a student from holding more than one current allocation.

The SQL solution applies joins, subqueries, filtering, sorting, GROUP BY, HAVING and aggregate functions to answer practical hostel-management questions. The realistic seed dataset makes the queries and execution-plan experiments more meaningful than a three-row demonstration database.

The indexing experiments showed that indexes must be chosen based on selectivity, query predicates, column order and table size. The attendance-date index changed a sequential scan to an index scan and produced a substantial speed-up in the documented experiment. In contrast, the skipped-meals index made execution slower, showing why indexes should be measured rather than added blindly.

Overall, the project achieved the intended database-design, SQL-query and performance-analysis outcomes and provides a foundation that could later be extended with payment transactions, hostel staff roles, room-transfer workflows, automated bill generation and additional reporting.

10. Declaration

I hereby declare that this case study report, “Hostel Room Allotment and Mess Billing Database Using SQL” (Case Study 44), represents my project work. The SQL database, schema, seed data, queries and execution-plan experiments were developed and executed in PostgreSQL, and the screenshots included in this report are based on the project execution environment.

I confirm that the report has been prepared for academic submission and that the database design and SQL implementation are presented for this case study.

Lokendra Joshi

Roll No.: 150096725062

Cohort: Larry Page

School of Future Tech, ITM Skills University